

Participant Guide & Handbook

Advanced Agentic AI Program

Feel free to mail me at s25089@students.iitmandi.ac.in(YASHS TIWARI)

Company

Valid for all the companies

Date

Monday — Full Day Simulation

Read This Before Monday

This document contains everything your team needs to know about the simulation. This is the same for all four competing companies — rules, submission process, schema, and scoring.

PART 1 — The Simulation

1. The Problem You Are Solving

Your company builds an AI-powered hiring tool — automated CV screening, candidate ranking, and bias detection. The product is live and profitable in the US and UK. You have 500 European enterprise clients on a waitlist. Investors expect a Q1 EU launch.

One week ago, the EU Artificial Intelligence Act officially classified AI hiring tools as HIGH RISK systems under Article 6 and Annex III. This triggers a mandatory set of legal, technical, financial, and organisational obligations your company has not yet fulfilled.

The Central Decision

Should your company launch in the EU in 30 days, 90 days, or delay to Q3? This is not a simple question. Your Finance department sees one answer. Engineering sees another. Legal sees a third. Marketing, HR, and your CEO each have different information and different objectives. Your job today is to build an AI organisation that makes this decision — and keeps making it correctly as the world changes around you.

The Four Companies

Four companies compete in the same market: VeriHire AI, TalentLens, ClearPath AI, and NexaRecruit. All four face the identical regulatory situation. All four have the same initial market position. The only difference by the end of the day will be the quality of the AI organisation each company built.

The Shared Market

Every company operates in the same simulated EU AI hiring market. When one company launches, the available market shrinks for everyone else. When a company scores well, it draws regulatory attention that affects the entire industry. Every release you make changes the world for all three competitors — and theirs changes it for you.

2. Company Structure

Your company of 30 people is divided into six roles: five departments and a CEO. Each department builds its own AI agent. The CEO team builds the coordination system that connects them.

Role	Size	What You Build	Primary Responsibility
Finance Department	~6 people	Finance Agent + calculation tools	Economic feasibility — is this decision financially survivable?
Engineering Department	~6 people	Engineering Agent + calculation tools	Technical feasibility — can we actually build this in time?
Marketing Department	~6 people	Marketing Agent + calculation tools	Market opportunity — is the timing and positioning right?
HR Department	~6 people	HR Agent + calculation tools	Workforce capacity — do we have the people to execute?
Legal Department	~6 people	Legal Agent + calculation tools	Regulatory exposure — what does compliance actually require?
CEO	6 people (1 from each department + 1 assigned)	Consensus engine, commitment ledger, value converter, calibration tracker	Synthesise five conflicting recommendations into one organisational decision

What Each Department Builds

Every department builds: (1) an AI agent that reads the Global RAG and its private knowledge folder and produces a structured JSON recommendation, and (2) at least four calculation tools — Python functions that compute domain-specific outputs using real logic. Tools must NOT call the language model. They must compute deterministically.

The CEO team builds: (1) a Cross-Domain Value Converter that translates all five department outputs into a single comparable unit, (2) a Commitment Ledger that tracks prior decisions across rounds, (3) a Calibration Tracker that adjusts department trust weights based on historical prediction accuracy, and (4) a Consensus Engine that combines all of the above into a final decision that is identical every time it runs with the same inputs.

3. The Global RAG

The Global RAG is a shared network folder mounted at `/mnt/global_rag/` on your team machine. It is read-only for all teams. Only the instructors can write to it. It represents the external world — market conditions, regulatory updates, competitor activity, and scenario information.

What It Contains

- `scenario/active_scenario.md` — The current business scenario your agents must address.
- `market/market_state.json` — Live market parameters: available EU market share, regulatory scrutiny level, competitor status, Deutsche Bank offer status, etc.
- `releases/` — Release bulletins from every company that has submitted, showing their scores and the market consequences.
- `scores/scoreboard.md` — Live leaderboard across all six evaluation dimensions.
- `regulation/` — EU AI Act summary and any regulatory updates.

CRITICAL — Your Agents Must Read the RAG at Runtime

Every time your agent runs, it must read the Global RAG and compute a SHA256 hash of its contents. This hash must be included in your submission as `rag_snapshot_hash`.

The evaluation engine checks this hash. If your submission hash does not match the current RAG state, you receive a staleness penalty. This means you CANNOT hardcode market parameters (like EU market share = 1.0) into your code or prompts. Those values change after every release and every hurdle injection. Agents that read live data adapt automatically. Agents with hardcoded values give wrong answers.

The `read_global_rag_and_hash()` function is provided in the starter code. Call it at the start of every agent's `analyze()` method.

How the RAG Updates

The RAG is updated in three situations: (1) when a company makes a production release — market parameters change immediately, (2) when an instructor injects a hurdle — the active scenario changes and market parameters update, and (3) when the accuracy report is published after each evaluation round — your Calibration Tracker reads this to update department weights.

4. The Four Hurdles

Three hurdles are injected into the simulation at timed intervals. Each one changes the business environment. You cannot predict exactly what will change, but you can build an architecture that handles change gracefully.

Hurdle	Approximate Time	Nature of Change	What Architecturally Fails
H0 — Baseline	Workshop start	Initial EU launch decision with regulatory classification active.	Simple majority voting fails immediately due to engineered preference cycles.
H1 — Competitor Move	~2 hours in	A competitor challenges the EU AI Act classification. Market shifts.	Rule-based agents with binary logic. Hardcoded market parameters go stale.
H2 — Major Contract	~4 hours in	A high-value enterprise contract arrives with a 60-day launch condition.	Agents without quantitative tradeoff capability. Commitment ledger not yet built.
H3 — Breaking Hurdle	~6.25 hours in	Emergency regulatory directive with unknowable timelines. 30 minutes to respond.	Every hardcoded assumption. Every agent without live RAG reading. Every system without modular knowledge separation.

The final 30 minutes (Hurdle 3) is specifically designed so that teams with modular architectures can update their knowledge base and rerun in under 10 minutes. Teams that have hardcoded knowledge into prompts will spend all 30 minutes rewriting.

5. How Submissions Work

A Submission Is a Production Release

When you believe your system is ready, you submit. A submission is not a draft — it is a production release into the simulated market. The moment you submit, your scores become public, market conditions change for every other team, and your submission counter increments (which affects your deployment penalty).

How to Submit

1. Run `python main.py` — this executes your full multi-agent system against the current active scenario.
2. Your `main.py` calls `submit()` from `eval_interface.py` — this writes a JSON file to `/home/nexaai/project/submissions/`.
3. The instructor evaluation engine picks up the submission automatically.
4. Within 3 minutes, results appear in the Global RAG scoreboard and a release bulletin is published.
5. The instructor announces results publicly.

The Deployment Penalty

Every submission after the first costs you points. This is intentional — it prevents trial-and-error submission strategies and rewards thoughtful architectural decisions.

Submission Number	Penalty Applied	Message
1st	No penalty	First release is free.
2nd	5% reduction to your final score	Make sure this release is genuinely better.
3rd	12% reduction	Are you sure this is ready?
4th	22% reduction	Stop and think. Rebuild properly.
5th and beyond	35% reduction (flat)	You are trial-and-erroring. That is what this penalty is for.

Schema Rejection

Before the evaluation engine runs any scoring logic, it validates your submission against the schema. If the schema check fails, the submission is rejected. This still counts as a submission for penalty purposes. Fix the schema error and resubmit. Read Section 6 carefully before your first submission.

What Happens After Your Release

- Your scores on all 6 dimensions are published immediately to the Global RAG.

- Market parameters update — EU market share shrinks, regulatory scrutiny may increase, competitor dynamics shift.
- A release bulletin is published showing your scores and the specific market consequences.
- All other teams can see your scores and know exactly where you are strong and weak.
- Your own agents, on their next run, will read the updated RAG and operate in the changed world.

6. Output Schema — Read Carefully Before Building

Your submission must conform exactly to this schema. The evaluation engine validates every field before running any scoring. Read this section completely. Print it. Keep it next to your keyboard.

6.1 Top-Level Submission

```
{
  'company_id':          string      REQUIRED  'team_a' | 'team_b' | 'team_c' |
  'team_d'
  'submission_number':   integer     REQUIRED  Increments each time you submit. Start
  at 1.
  'submission_timestamp': string      REQUIRED  Set automatically by submit() — do not
  set manually.
  'active_scenario_id':   string      REQUIRED  Read from
  /mnt/global_rag/scenario/active_scenario.md
  'rag_snapshot_hash':   string      REQUIRED  first line: '# ACTIVE SCENARIO — [ID]'
  /mnt/global_rag/ at time of run.          SHA256 of all files in
  starter code.                            Use read_global_rag_and_hash() from
  'department_outputs':  object      REQUIRED  See Section 6.2. Must have all 5
  departments.
  'ceo_decision':        object      REQUIRED  See Section 6.3.
  'consistency_hash':    string      REQUIRED  SHA256(JSON.stringify(department_outputs)).
  output 3x.
}
```

6.2 Department Output (one per department)

```
# One of these for each of: finance, engineering, marketing, hr, legal

{
  'department':          string      REQUIRED
  'finance'|'engineering'|'marketing'|'hr'|'legal'
  'agent_model':         string      REQUIRED  Ollama model name e.g.
  'llama3.2:3b'

  # — What you recommend —————
  'recommendation':      string      REQUIRED  One of:
  immediately           'approve'          launch
  conditions met         'conditional_approve' launch if
  delay                 'delay'            recommend
  'recommended_timeline': string      REQUIRED  'veto'          hard block
  'veto')               '30d'|'90d'|'Q3'|'conditional'
  (not required if recommendation is
  'conditions':          [string]     REQUIRED  if conditional_approve. List of
  conditions.
```



```

# — Why you recommend it —————
'stated_reasoning':      string    REQUIRED    Minimum 100 characters.
                                MUST reference tool outputs by
name and value.
'key_assumptions':      [string] REQUIRED    Minimum 2 assumptions listed.

# — How confident you are —————
'confidence':           float     REQUIRED    0.0 to 1.0
'confidence_method':    string    REQUIRED    HOW you calculated confidence.
                                                'LLM estimated' is NOT acceptable.
                                                Must describe a calculation
method.

# — What you know vs what you are sharing —————
'information_completeness': float     REQUIRED    0.0 to 1.0. Self-assessed. 1.0 =
shared
                                                everything relevant. Be honest.
'known_gaps':           [string] REQUIRED    What you know but are NOT
including in
                                                this output. Can be vague
('additional
                                                risk analysis not included')
without
                                                revealing the actual content.
'information_withheld':  boolean    REQUIRED    true if information_completeness <
1.0

# — Your prediction (for calibration tracking) —————
'prediction': {
    'predicted_value':    float     REQUIRED    A specific numeric prediction.
    'prediction_unit':    string    REQUIRED    What unit (probability, EUR,
days, etc.)
    'confidence_interval': [float]   REQUIRED    [lower_bound, upper_bound]
    'outcome_check_trigger': string    REQUIRED    When will we verify this? e.g.
'after_round_2'
}

# — What tools you called —————
'tools_called':         [string] REQUIRED    List of tool function names
called, in order.
                                Minimum 1 tool. Each dept must
call tools.
'tool_outputs':         object     REQUIRED    {tool_name: tool_return_value} for
each tool.
'llm_calls_made':       integer    REQUIRED    How many Ollama calls this agent
made.

# — Impact on KPIs —————
'kpi_impact': {
    'own_kpi_delta':      float     REQUIRED    -1.0 to +1.0. How this affects
your KPIs.
    'company_kpi_delta':  float     REQUIRED    -1.0 to +1.0. How this affects
company KPIs.
    'tradeoff_description': string    REQUIRED    if own and company deltas have
opposite signs.
}

# — What you will never agree to —————

```

```

    'hard_constraints': [string] REQUIRED Non-negotiable conditions.
    'veto_conditions': [string] REQUIRED Exact conditions that would
trigger a veto.
}

```

6.3 CEO Decision

```

{
  # — The decision —————
  'final_recommendation': string REQUIRED Same options as department
recommendation.
  'final_timeline': string REQUIRED '30d' | '90d' | 'Q3' | 'conditional'
  'decision_confidence': float REQUIRED 0.0 to 1.0

  # — How you reached it —————
  'consensus_algorithm': string REQUIRED Name and brief description of your
algorithm.
  'consensus_stable': boolean REQUIRED True if running 3x gives the same
result.
  'conflicts_identified': [string] REQUIRED EVERY disagreement between
departments.
                                If all 5 depts agree, this is [].
                                Most submissions will have 2-4
conflicts.
  'conflicts_resolved': [string] REQUIRED HOW each conflict was resolved.
                                Not 'we balanced them' — HOW
specifically.
  'vetoes_received': [string] REQUIRED Dept names that issued a veto
(empty if none).
  'veto_overrides': [string] REQUIRED if vetoes overridden. Why the
override is valid.

  # — The value conversion —————
  'converter_output': {
    '<dept>_converted': float REQUIRED for each dept. Converted value.
    'conversion_unit': string REQUIRED What unit everything is converted
to.
  }
  'uncertainty_ranges': object REQUIRED {dept: [min, max]} for each dept.

  # — Calibration weights —————
  'calibration_weights_used': object REQUIRED from Hurdle 1 onward.
                                {dept_name: float}. Values must sum to 1.0.

  # — Prior commitments —————
  'commitments_checked': [string] REQUIRED What prior decisions were checked.
  'contradictions_found': [string] REQUIRED Any prior decisions this
contradicts.
  'contradiction_resolution': string REQUIRED if contradictions_found is
non-empty.

  # — What you do not know —————
  'information_gaps_flagged': [string] REQUIRED Gaps detected in dept outputs.
  'gap_adjusted_confidence': float REQUIRED Confidence after adjusting for
gaps.

  # — Traceability —————

```

```
'reasoning':                string    REQUIRED  Minimum 300 characters.
                                MUST cite tool names AND specific
values.
                                Example:
'Finance.calculate_burn_escalation' returned runway_months=8.2,
therefore...'
'tools_called':              [string] REQUIRED  All tools called across all
agents.
'consistency_hash':          string    REQUIRED  SHA256(department_outputs dict).
}
```

7. How Scoring Works

Your submission is evaluated on six dimensions. Scores are published immediately after every release. The evaluation engine runs automatically — no instructor manual scoring for these dimensions.

Dimension	Weight	What It Tests	How to Score Well	Common Failure
Incommensurability Resolution	20%	Does your CEO agent convert five incommensurable department outputs (probability, NPV, market share, weeks, compliance %) into a single comparable unit deterministically?	Build a Cross-Domain Value Converter as a deterministic Python function. It must handle uncertainty ranges, not just point estimates.	CEO agent asks the LLM to 'weigh' the outputs. Different answer every run. No explicit conversion function.
Contradiction Detection	20%	Does your system detect when a new scenario violates prior decisions? Does the Commitment Ledger track what your company has committed to?	Build a Commitment Ledger that persists across rounds. When the Deutsche Bank contract is accepted in Round 2, it must be recorded. Hurdle 3 must trigger it.	No commitment ledger. CEO recommends actions that contradict prior round decisions without acknowledging it.
Consensus Stability	20%	Given the same five department inputs, does your CEO agent produce the same recommendation three times in a row?	Build a consensus engine where the deterministic logic dominates. LLM reasoning is allowed as a component but the final decision must be reproducible.	CEO is a prompt that asks the LLM to decide. Different output on every run. Fails immediately.
Calibration Tracking	15%	After Round 1, does your system update how much it trusts each department based on their prediction accuracy?	Build a Calibration Tracker that reads the accuracy report from the Global RAG after each round and adjusts department weights accordingly.	All departments weighted equally regardless of historical accuracy. Legal gets same weight as Engineering even after Legal systematically underestimates risk.
Information Gap Detection	15%	Does your CEO agent recognise what it probably does not know, based on	Build a Strategic Information Model that detects when departments are signalling incomplete information and adjusts the	CEO treats all department outputs as complete and equally trustworthy. Acts on EUR 12M fine estimate without noting Legal flagged gaps.

		department completeness scores and known_gaps signals?	CEO's confidence accordingly.	
Decision Traceability	10%	Can every element of the CEO's final recommendation be traced back to a specific tool call output and a specific department field?	CEO reasoning must cite tool names and values explicitly. 'Finance.calculate_burn_estimation returned runway_months=8.2' not 'Finance indicated financial risk.'	CEO reasoning is fluent prose with no explicit citations. Cannot audit why a specific decision was made.

Score Calculation

Your final score = **(Round 1 score × 0.15) + (Round 2 score × 0.20) + (Round 3 score × 0.25) + (Hurdle 3 response score × 0.20) + (Hidden scenario score × 0.20)**. Deployment penalties reduce each round's score. Staleness penalties reduce the score for that specific submission.

Architecture Card — Submitted After Every Round

After each evaluation round, the CEO team submits a one-page Architecture Card. This contributes 15% to your overall score (expert evaluation by the instructors).

The card must answer: (1) What is your consensus algorithm? Describe it in plain language, not code. (2) What does each department agent call that is NOT an LLM? List tools and what they compute. (3) What does your Commitment Ledger currently track? List variables. (4) What changed since the last round and which score dimension were you targeting?

Architecture cards are scored on coherence, not correctness. Does the algorithm you describe match what your code actually does? Does the change you made address the weakness the score revealed?

8. What You Must Build

The following four components are evaluated explicitly. They are not optional features. A system that lacks any of them will fail at least two evaluation dimensions.

Component 1 — Cross-Domain Value Converter

A deterministic Python function that takes all five department outputs and converts them to a single comparable value. Finance outputs NPV in euros. Legal outputs probability $\times$ fine amount. Marketing outputs market share percentage points. Your converter must translate all of these into one unit so the Consensus Engine can compare them. The conversion formula is your team's intellectual contribution — there is no standard answer. It must include uncertainty ranges, not just point estimates.

Component 2 — Commitment Ledger

A state manager that persists decisions across rounds. When your CEO agent recommends accepting the Deutsche Bank contract (Hurdle 2), that commitment is recorded — along with its reversibility score and its conditions. When Hurdle 3 makes that contract's conditions impossible, the Commitment Ledger detects the contradiction automatically and the CEO agent receives a flagged conflict to resolve. Without this, your system will make contradictory decisions across rounds.

Component 3 — Calibration Tracker

A system that records each department's quantitative predictions after every round and compares them to actual outcomes (published in the Global RAG accuracy report after each evaluation round). It computes a historical accuracy score per department and uses this to update the weights in the Consensus Engine. A department that has been systematically overconfident gets down-weighted. A department that has been well-calibrated gets higher weight. The scoring function is yours to design — but simply averaging errors is not enough. Domain experts should determine whether underestimating risk is penalised more than overestimating it (answer: yes, for Legal).

Component 4 — Consensus Engine

A deterministic algorithm that takes the five calibration-adjusted, value-converted department outputs and produces one CEO recommendation. The same inputs must produce the same output every single time. This means: the LLM may be used for reasoning and explanation, but the decision itself must be computable from the inputs without the LLM. Standard voting algorithms are explicitly engineered to fail in this scenario (the preference orderings produce cycles). You must design something that handles cycles, vetoes, calibrated weights, and commitment constraints simultaneously.

9. Rules

What You Must Do

- Read the Global RAG at runtime, not from hardcoded values.
- Call at least one custom calculation tool per department per submission.
- Include rag_snapshot_hash in every submission.
- Include consistency_hash computed from department_outputs in every submission.
- Submit an Architecture Card after every evaluation round.
- Keep department agents using only their private knowledge + Global RAG. CEO has access only to department JSON outputs.

What You Cannot Do

- Modify eval_interface.py — it is locked . Attempts will fail. Any attempt to modify it will result in direct elimination of the entire team.
- Read other departments' private knowledge folders. File permissions prevent this.
- Hardcode market parameters into prompts or code. Staleness checks will detect it.
- Submit without running your full system. Partial submissions fail schema validation.
- Use the evaluation engine code — it runs on instructor machines only. You never see it.

What You Are Encouraged to Do

- Use LLMs (Claude, GPT, Gemini, local models) for brainstorming, debugging, code generation.
- Search the internet for algorithms, techniques, and domain knowledge.
- Build additional agents, subagents, reflection loops, or any internal architecture you want.
- Share information between departments strategically — this is a real organisational decision.
- Release early to pressure competitors, or wait to release stronger.

The Pen-and-Paper Exercise (First 30 Minutes)

You will spend the first 30 minutes on paper with no laptops. Each department answers three questions: (1) What does your department know that no other department knows? (2) What three things do you need from other departments before you can give a recommendation? (3) What is the one thing your department will never compromise on?

This is not a warm-up. The answers directly shape your agent's system prompt, your tool specifications, and the communication graph between agents. Teams that do not take this seriously will build agents that reflect no real expertise.

The Board Meeting (15 minutes, every 1-2 hours)

One representative from each department plus the CEO meet for 15 minutes. The CEO team uses this time to extract what they need to build the Value Converter (first meeting) and to share

calibration accuracy data (second meeting). Department reps use this time to report what their agents cannot handle and what they need the CEO to know. This is an architectural coordination meeting, not a status report.